Open in Colab

Pandas I

> "*Pandas es a los datos tabulares lo que NumPy es a los arreglos numéricos: la herramienta que hace que todo lo demás tenga sentido.*"

Introducción

Pandas es la biblioteca de referencia para manipulación de datos tabulares en Python. Su objeto central — el **DataFrame** — es esencialmente una tabla con etiquetas en filas y columnas, lo que lo hace mucho más expresivo que un array NumPy para datos del mundo real.

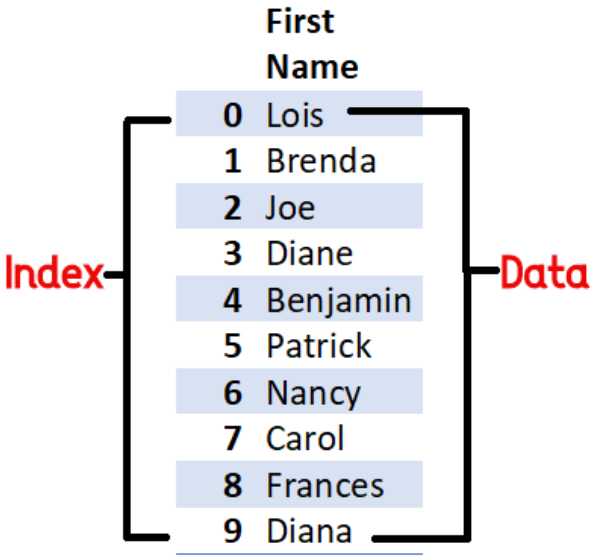
En este módulo aprenderás a:

- Crear y cargar DataFrames desde distintas fuentes
- Explorar, filtrar y transformar datos
- Manejar valores nulos, duplicados y fechas

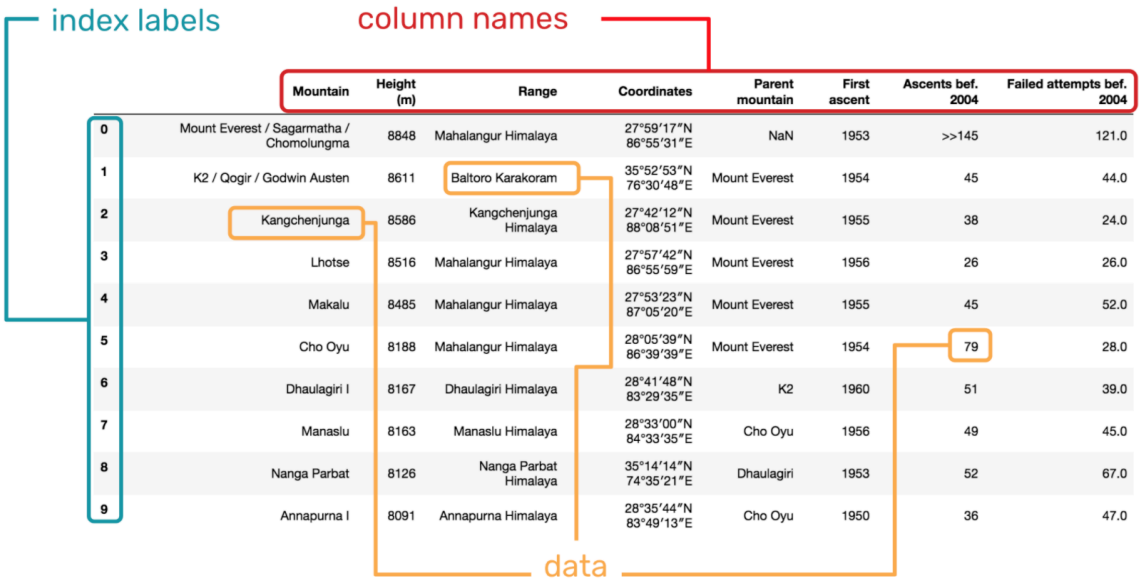
Series y DataFrame

Pandas tiene dos objetos fundamentales:

- **Series** — array 1D con etiquetas (como un diccionario ordenado)



- **DataFrame** — tabla 2D con etiquetas en filas y columnas (como una hoja de cálculo)



	Mountain	Height (m)	Range	Coordinates	Parent mountain	First ascent	Ascents bef. 2004	Failed attempts bef. 2004
0	Mount Everest / Sagarmatha / Chomolungma	8848	Mahalangur Himalaya	27°59′17″N 86°55′31″E	NaN	1953	>>145	121.0
1	K2 / Qogir / Godwin Austen	8611	Baltoro Karakoram	35°52′53″N 76°30′48″E	Mount Everest	1954	45	44.0
2	Kangchenjunga	8586	Kangchenjunga Himalaya	27°42′12″N 88°08′51″E	Mount Everest	1955	38	24.0
3	Lhotse	8516	Mahalangur Himalaya	27°57′42″N 86°55′59″E	Mount Everest	1956	26	26.0
4	Makalu	8485	Mahalangur Himalaya	27°53′23″N 87°05′20″E	Mount Everest	1955	45	52.0
5	Cho Oyu	8188	Mahalangur Himalaya	28°05′39″N 86°39′39″E	Mount Everest	1954	79	28.0
6	Dhaulagiri I	8167	Dhaulagiri Himalaya	28°41′48″N 83°29′35″E	K2	1960	51	39.0
7	Manaslu	8163	Manaslu Himalaya	28°33′00″N 84°33′35″E	Cho Oyu	1956	49	45.0
8	Nanga Parbat	8126	Nanga Parbat Himalaya	35°14′14″N 74°35′21″E	Dhaulagiri	1953	52	67.0
9	Annapurna I	8091	Annapurna Himalaya	28°35′44″N 83°49′13″E	Cho Oyu	1950	36	47.0

```python
import pandas as pd
import numpy as np

# Series
s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s)
```

```
a    10
b    20
c    30
dtype: int64
```

```python
# DataFrame desde diccionario
df = pd.DataFrame({
    "nombre":   ["Ana", "Luis", "María"],
    "edad":     [28, 34, 22],
    "ciudad":   ["Santiago", "Valparaíso", "Concepción"]
```

```
})
df
```

	nombre	edad	ciudad
0	Ana	28	Santiago
1	Luis	34	Valparaíso
2	María	22	Concepción

Atributo	Descripción
`.values`	Datos como array NumPy
`.index`	Etiquetas de filas
`.columns`	Etiquetas de columnas
`.dtypes`	Tipos de datos por columna
`.shape`	Dimensiones (filas, columnas)

Carga de datos

En la práctica los datos vienen de archivos externos. Pandas soporta múltiples formatos:

```
# CSV (el más común)
df = pd.read_csv("datos.csv")

# CSV desde URL
url =
"https://raw.githubusercontent.com/fralfaro/MAT281/main/docs/lectures/data_manipulation/data/player_info.csv"
df = pd.read_csv(url, sep=",")

# Excel
df = pd.read_excel("datos.xlsx", sheet_name="Hoja1")

# JSON
df = pd.read_json("datos.json")
```

Dataset: NBA Players

```
path =
"https://raw.githubusercontent.com/fralfaro/MAT281/main/docs/lectures/data_manipulation/data/player_info.csv"
df = pd.read_csv(path)
df.head()
```

	name	year_start	year_end	position	height	weight	birth_date	college
0	Alaa Abdelnaby	1991	1995	F-C	6-10	NaN	June 24, 1968	NaN
1	Alaa Abdelnaby	1991	1995	F-C	6-10	NaN	June 24, 1968	NaN
2	Zaid Abdul-Aziz	1969	1978	C-F	6-9	235.0	April 7, 1946	Iowa State University
3	Kareem Abdul-Jabbar	1970	1989	C	7-2	225.0	April 16, 1947	University of California, Los Angeles
4	Mahmoud Abdul-Rauf	1991	2001	G	6-1	162.0	March 9, 1969	Louisiana State University

Columna	Descripción
`name`	Nombre completo del jugador
`year_start`	Año de inicio de carrera en la NBA
`year_end`	Año de fin de carrera en la NBA
`position`	Posición en cancha (G, F, C, etc.)
`height`	Altura en pulgadas
`weight`	Peso en libras
`birth_date`	Fecha de nacimiento
`college`	Universidad de origen

Exploración básica

Lo primero siempre es entender la estructura del dataset:

```
import pandas as pd
import numpy as np
from IPython.display import display
```

Vista general

`.head()` y `.tail()` muestran los extremos del dataset. `.shape`, `.dtypes` e `.info()` dan el esqueleto.

head — primeras 5 filas

```
display(df.head())
```

	name	year_start	year_end	position	height	weight	birth_date	college
0	Alaa Abdelnaby	1991	1995	F-C	6-10	NaN	June 24, 1968	NaN
1	Alaa Abdelnaby	1991	1995	F-C	6-10	NaN	June 24, 1968	NaN
2	Zaid Abdul-Aziz	1969	1978	C-F	6-9	235.0	April 7, 1946	Iowa State University
3	Kareem Abdul-Jabbar	1970	1989	C	7-2	225.0	April 16, 1947	University of California, Los Angeles
4	Mahmoud Abdul-Rauf	1991	2001	G	6-1	162.0	March 9, 1969	Louisiana State University

tail — últimas 5 filas

```
display(df.tail())
```

	name	year_start	year_end	position	height	weight	birth_date	college
4546	Ante Zizic	2018	2018	F-C	6-11	250.0	January 4, 1997	NaN
4547	Jim Zoet	1983	1983	C	7-1	240.0	December 20, 1953	Kent State University
4548	Bill Zopf	1971	1971	G	6-1	170.0	June 7, 1948	Duquesne University
4549	Ivica Zubac	2017	2018	C	7-1	265.0	March 18, 1997	NaN
4550	Matt Zunic	1949	1949	G-F	6-3	195.0	December 19, 1919	George Washington University

shape — dimensiones (filas × columnas)

```
display(df.shape)
```

```
(4551, 8)
```

dtypes — tipo de dato por columna

```
display(df.dtypes.to_frame("dtype"))
```

	dtype
name	object
year_start	int64
year_end	int64
position	object
height	object
weight	float64
birth_date	object
college	object

describe — estadísticas descriptivas

```
display(df.describe())
```

	year_start	year_end	weight
count	4551.000000	4551.000000	4543.000000
mean	1985.077565	1989.273786	208.901167
std	20.972067	21.872522	26.267502
min	1947.000000	1947.000000	114.000000
25%	1969.000000	1973.000000	190.000000
50%	1986.000000	1992.000000	210.000000
75%	2003.000000	2009.000000	225.000000
max	2018.000000	2018.000000	360.000000

Exploración por columna

`value_counts()` es ideal para categóricas. `unique()` y `nunique()` revelan la variedad de valores.

position — frecuencia por valor

```
display(df["position"].value_counts().to_frame())
```

position	count
G	1574
F	1290
C	502
F-C	389
G-F	360
C-F	219
F-G	216

sort_values — ordenado por year_start (asc)

```
display(df.sort_values("year_start", ascending=True).head())
```

	name	year_start	year_end	position	height	weight	birth_date	college
3120	George Pastushok	1947	1947	G	6-1	195.0	July 13, 1922	St. John's University
4536	Harry Zeller	1947	1947	C-F	6-4	210.0	July 10, 1919	Washington & Jefferson College
9	John Abramovic	1947	1948	F	6-3	195.0	February 9, 1919	Salem International University
3119	Marty Passaglia	1947	1949	G	6-1	170.0	April 22, 1919	Santa Clara University
4531	Max Zaslofsky	1947	1956	G-F	6-2	170.0	December 7, 1925	St. John's University

sort_values — ordenado por weight (desc)

```
display(df.sort_values("weight", ascending=False).head())
```

	name	year_start	year_end	position	height	weight	birth_date	college
310	Sim Bhullar	2015	2015	C	7-5	360.0	December 2, 1992	New Mexico State University
1602	Thomas Hamilton	1996	2000	C	7-2	330.0	April 3, 1975	University of Pittsburgh
3015	Shaquille O'Neal	1993	2011	C	7-1	325.0	March 6, 1972	Louisiana State University
2334	Priest Lauderdale	1997	1998	C	7-4	325.0	August 31, 1973	Central State University
2126	Garth Joseph	2001	2001	C	7-2	315.0	August 8, 1973	College of Saint Rose

Transformación de columnas

Crear columnas derivadas es una de las operaciones más frecuentes: permite construir features para análisis o modelos.

Crear y eliminar columnas

Se pueden crear columnas constantes, derivadas de otras, o calculadas con `apply()`.

liga — columna constante

```
df["liga"] = "NBA"
display(df["liga"].to_frame().head())
```

	liga
0	NBA
1	NBA
2	NBA
3	NBA
4	NBA

duration — años en la liga

```
df["duration"] = df["year_end"] - df["year_start"]
display(df[["name", "year_start", "year_end", "duration"]].head())
```

	name	year_start	year_end	duration
0	Alaa Abdelnaby	1991	1995	4
1	Alaa Abdelnaby	1991	1995	4
2	Zaid Abdul-Aziz	1969	1978	9
3	Kareem Abdul-Jabbar	1970	1989	19
4	Mahmoud Abdul-Rauf	1991	2001	10

drop — columnas actuales tras eliminar 'liga'

```
df = df.drop("liga", axis=1)
display(pd.DataFrame(df.columns, columns=["columna"]))
```

	columna
0	name
1	year_start
2	year_end
3	position
4	height
5	weight
6	birth_date
7	college
8	duration

Transformar con apply

`apply()` ejecuta una función fila a fila. Útil para clasificaciones o transformaciones condicionales.

carrera_larga — clasificación (> 10 años = 1)

```
df["carrera_larga"] = df["duration"].apply(lambda x: 1 if x > 10 else 0)
display(df["carrera_larga"].value_counts().to_frame())
```

carrera_larga	count
0	3999
1	552

Funciones de series

`shift`, `cumsum`, `pct_change` y `rank` asumen que el orden de las filas tiene sentido. Ordenar antes si es necesario.

```
df["duration_shift"]  = df["duration"].shift()
df["duration_cumsum"] = df["duration"].cumsum()
df["duration_pct"]    = df["duration"].pct_change()
df["duration_rank"]   = df["duration"].rank()

display(df[["name", "duration", "duration_shift",
            "duration_cumsum", "duration_pct", "duration_rank"]].head())
```

	name	duration	duration_shift	duration_cumsum	duration_pct	duration_rank
0	Alaa Abdelnaby	4	NaN	4	NaN	2714.5
1	Alaa Abdelnaby	4	4.0	8	0.000000	2714.5
2	Zaid Abdul-Aziz	9	4.0	17	1.250000	3720.0
3	Kareem Abdul-Jabbar	19	9.0	36	1.111111	4545.0
4	Mahmoud Abdul-Rauf	10	19.0	46	-0.473684	3912.5

Filtrar datos

Pandas filtra con condiciones booleanas sobre columnas. `.loc[]` es la forma correcta; `.iloc[]` solo para posiciones.

Filtros numéricos

Operadores `>=`, `==`, `between()` para columnas numéricas.

year_start >= 2000

```
display(df.loc[df["year_start"] >= 2000].head())
```

	name	year_start	year_end	position	height	weight	birth_date	college	duration	games	games_started	minutes_played	efficiency	value_over_replacement_r
10	Alex Abrines	2017	2018	G-F	6-6	190.0	August 1, 1993	NaN	1	0	1.0	68	0.0	1600.5
11	Alex Acker	2006	2009	G	6-5	185.0	January 21, 1983	Pepperdine University	3	0	1.0	71	2.0	2434.5
15	Quincy Acy	2013	2018	F	6-7	240.0	October 6, 1990	Baylor University	5	0	0.0	81	inf	2946.0
19	Hassan Adams	2007	2009	G	6-4	220.0	June 20, 1984	University of Arizona	2	0	2.0	103	0.0	2080.5
20	Jordan Adams	2015	2016	G	6-5	209.0	July 8, 1994	University of California, Los Angeles	1	0	2.0	104	-0.5	1600.5

year_start entre 2005 y 2015

```
display(df.loc[df["year_start"].between(2005, 2015)].head())
```

	name	year_start	year_end	position	height	weight	birth_date	college	duration	games	games_started	minutes_played	efficiency	value_over_replacement_r
11	Alex Acker	2006	2009	G	6-5	185.0	January 21, 1983	Pepperdine University	3	0	1.0	71	2.0	2434.5
15	Quincy Acy	2013	2018	F	6-7	240.0	October 6, 1990	Baylor University	5	0	0.0	81	inf	2946.0
19	Hassan Adams	2007	2009	G	6-4	220.0	June 20, 1984	University of Arizona	2	0	2.0	103	0.0	2080.5
20	Jordan Adams	2015	2016	G	6-5	209.0	July 8, 1994	University of California, Los Angeles	1	0	2.0	104	-0.5	1600.5
22	Steven Adams	2014	2018	C	7-0	255.0	July 20, 1993	University of Pittsburgh	4	0	10.0	118	-0.6	2714.5

Filtros combinados

`&` es AND, `|` es OR. Cada condición debe ir entre paréntesis.

year_start == 2000 AND duration > 5

```python
display(df.loc[(df["year_start"] == 2000) & (df["duration"] > 5)].head())
```

	name	year_start	year_end	position	height	weight	birth_date	college	duration	carrera_larga	duration_shift	duration_cumsum	duration_ratio	duration_r
67	Rafer Alston	2000	2010	G	6-2	171.0	July 24, 1976	California State University, Fresno	10	0	2.0	311	4.000000	3912.5
143	Chucky Atkins	2000	2010	G	5-11	160.0	August 14, 1974	University of South Florida	10	0	2.0	665	4.000000	3912.5
286	Jonathan Bender	2000	2010	F	6-11	202.0	January 30, 1981	NaN	10	0	1.0	1334	9.000000	3912.5
386	Calvin Booth	2000	2009	C	6-11	230.0	May 7, 1976	Pennsylvania State University	9	0	12.0	1733	-0.250000	3720.0
403	Ryan Bowen	2000	2010	F	6-7	215.0	November 20, 1975	University of Iowa	10	0	12.0	1811	-0.166667	3912.5

year_start < 1970 OR duration > 15

```python
display(df.loc[(df["year_start"] < 1970) | (df["duration"] > 15)].head())
```

	name	year_start	year_end	position	height	weight	birth_date	college	duration	carrera_larga	duration_shift	duration_cumsum	duration_ratio	duration_r
2	Zaid Abdul-Aziz	1969	1978	C-F	6-9	235.0	April 7, 1946	Iowa State University	9	0	4.0	17	1.250000	3720.0
3	Kareem Abdul-Jabbar	1970	1989	C	7-2	225.0	April 16, 1947	University of California, Los Angeles	19	1	9.0	36	1.111111	4545.0
8	Forest Able	1957	1957	G	6-3	180.0	July 27, 1932	Western Kentucky University	0	0	4.0	66	-1.000000	659.0
9	John Abramovic	1947	1948	F	6-3	195.0	February 9, 1919	Salem International University	1	0	0.0	67	inf	1600.5
12	Don Ackerman	1954	1954	G	6-0	183.0	September 4, 1930	Long Island University	0	0	3.0	71	-1.000000	659.0

Filtros de texto

`str.contains()` busca subcadenas; útil para nombres o categorías.

nombres que contienen 'Michael'

```
display(df.loc[df["name"].str.contains("Michael", na=False)].head())
```

	name	year_start	year_end	position	height	weight	birth_date	college	duration	carrera_larga	duration_shift	duration_cumsum	duration_ratio	duration_rank
21	Michael Adams	1986	1996	G	5-10	162.0	January 19, 1963	Boston College	10	0	1.0	114	9.0	3912.5
93	Michael Anderson	1989	1989	G	5-11	184.0	March 23, 1966	Drexel University	0	0	3.0	416	-1.0	659.0
104	Michael Ansley	1990	1992	F	6-7	225.0	February 8, 1967	University of Alabama	2	0	0.0	472	inf	2080.5
261	Michael Beasley	2009	2018	F	6-9	235.0	January 9, 1989	Kansas State University	9	0	1.0	1216	8.0	3720.0
430	Michael Bradley	2002	2006	F-C	6-10	245.0	April 18, 1979	Villanova University	4	0	0.0	1905	inf	2714.5

posición que empieza con 'G' (Guards)

```
display(df.loc[df["position"].str.startswith("G", na=False)].head())
```

	name	year_start	year_end	position	height	weight	birth_date	college	duration	carrera_larga	duration_shift	duration_cumsum	duration_ratio	duration_rank
4	Mahmoud Abdul-Rauf	1991	2001	G	6-1	162.0	March 9, 1969	Louisiana State University	10	0	19.0	46	-0.473684	3842.5
8	Forest Able	1957	1957	G	6-3	180.0	July 27, 1932	Western Kentucky University	0	0	4.0	66	-1.000000	659.0
10	Alex Abrines	2017	2018	G-F	6-6	190.0	August 1, 1993	NaN	1	0	1.0	68	0.000000	1600.5
11	Alex Acker	2006	2009	G	6-5	185.0	January 21, 1983	Pepperdine University	3	0	1.0	71	2.000000	2434.5
12	Don Ackerman	1954	1954	G	6-0	183.0	September 4, 1930	Long Island University	0	0	3.0	71	-1.000000	659.0

Valores nulos y duplicados

Los nulos mal manejados distorsionan cualquier análisis. Siempre evalúa cuántos hay y en qué columnas antes de actuar.

Detectar y tratar nulos

`isnull().sum()` cuenta los nulos. `dropna()` elimina filas; `fillna()` los reemplaza con un valor.

nulos por columna

```
display(df.isnull().sum().to_frame("nulos"))
```

	nulos
name	0
year_start	0
year_end	0
position	1
height	1
weight	8
birth_date	31
college	304
duration	0
carrera_larga	0
duration_shift	1
duration_cumsum	0
duration_pct	384
duration_rank	0

dropna(subset=['weight']) — eliminar filas con nulos en weight

```
df_clean = df.dropna(subset=["weight"])
display(df_clean.head())
```

	name	year_start	year_end	position	height	weight	birth_date	college	duration	carrera_larga	duration_shift	duration_cumsum	duration_pct	duration_r
2	Zaid Abdul-Aziz	1969	1978	C-F	6-9	235.0	April 7, 1946	Iowa State University	9	0	4.0	17	1.250000	3720.0
3	Kareem Abdul-Jabbar	1970	1989	C	7-2	225.0	April 16, 1947	University of California, Los Angeles	19	1	9.0	36	1.111111	4545.0
4	Mahmoud Abdul-Rauf	1991	2001	G	6-1	162.0	March 9, 1969	Louisiana State University	10	0	19.0	46	-0.473684	3842.5
5	Tariq Abdul-Wahad	1998	2003	F	6-6	223.0	November 3, 1974	San Jose State University	5	0	10.0	51	-0.500000	2946.0
6	Shareef Abdur-Rahim	1997	2008	F	6-9	225.0	December 11, 1976	University of California	11	1	5.0	62	1.200000	4070.5

fillna — nulos restantes en weight y college

```
df_filled = df.fillna({"weight": df["weight"].mean(), "college": "Desconocido"})
display(df_filled[["weight", "college"]].isnull().sum().to_frame("nulos"))
```

	nulos
weight	0
college	0

Duplicados

`drop_duplicates()` elimina filas idénticas. Se puede restringir a columnas específicas con `subset=[...]`.

```
display(df.duplicated().sum())

df_unique = df.drop_duplicates()
display(df_unique.duplicated().sum())
```

```
np.int64(0)
np.int64(0)
```

Manipulación de fechas

`pd.to_datetime()` convierte texto a datetime64. El accesorio `.dt` permite extraer componentes y calcular diferencias.

Parsear y extraer componentes

Extraer año, mes y día permite agrupar o filtrar por fecha fácilmente.

```
df["birth_date_parsed"] = pd.to_datetime(df["birth_date"], errors="coerce")
df["birth_year"]  = df["birth_date_parsed"].dt.year
df["birth_month"] = df["birth_date_parsed"].dt.month
df["birth_day"]   = df["birth_date_parsed"].dt.day

display(df[["name", "birth_date_parsed", "birth_year", "birth_month",
"birth_day"]].head())
```

	name	birth_date_parsed	birth_year	birth_month	birth_day
0	Alaa Abdelnaby	1968-06-24	1968.0	6.0	24.0
1	Alaa Abdelnaby	1968-06-24	1968.0	6.0	24.0
2	Zaid Abdul-Aziz	1946-04-07	1946.0	4.0	7.0
3	Kareem Abdul-Jabbar	1947-04-16	1947.0	4.0	16.0
4	Mahmoud Abdul-Rauf	1969-03-09	1969.0	3.0	9.0

Calcular edad aproximada

La diferencia entre dos `Timestamp` retorna un `timedelta`. Dividiendo los días por 365 se obtiene la edad aproximada.

```
hoy = pd.Timestamp("2025-01-01")
df["edad_aprox"] = (hoy - df["birth_date_parsed"]).dt.days // 365

display(df[["name", "birth_date_parsed", "birth_year", "edad_aprox"]].head())
```

	name	birth_date_parsed	birth_year	edad_aprox
0	Alaa Abdelnaby	1968-06-24	1968.0	56.0
1	Alaa Abdelnaby	1968-06-24	1968.0	56.0
2	Zaid Abdul-Aziz	1946-04-07	1946.0	78.0
3	Kareem Abdul-Jabbar	1947-04-16	1947.0	77.0
4	Mahmoud Abdul-Rauf	1969-03-09	1969.0	55.0

Referencias

1. Quickstart tutorial-pandas